febbraio 2026

esercizio 1

```
//Variabili condivise
int a=0;
```

```
--Process P
     while(true){
p1:    Sezione Non Critica
p2:    a=1;
p3:    if(a!=1){
p4:      a=2;}
p5:    while(a!=1){}
p6:    Sezione Critica
p7:    a=2;
     }
```

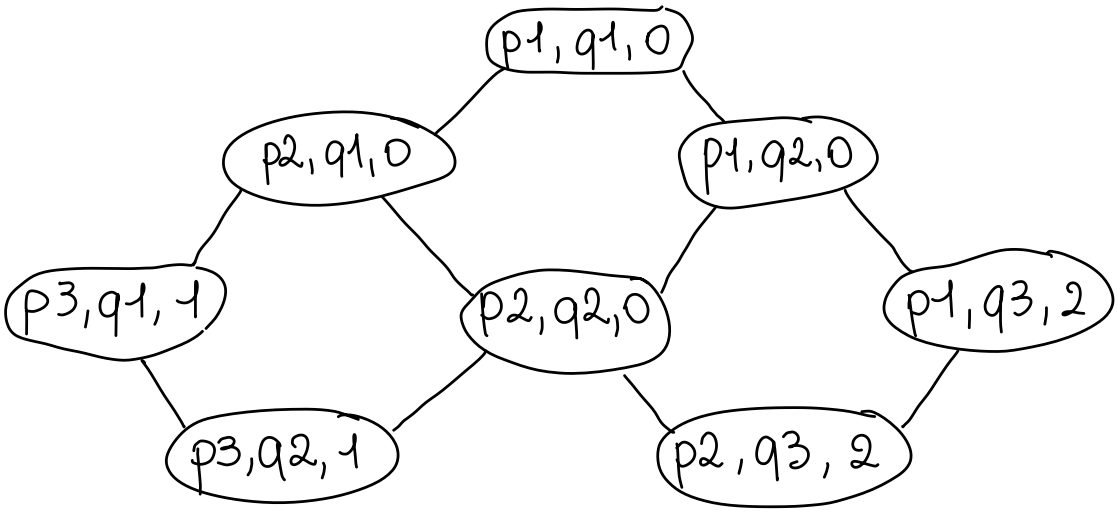
```
--Process Q
     while(true){
q1:    Sezione Non Critica
q2:    a=2;
q3:    if(a!=2){
q4:      a=1;}
q5:    while(a!=2){}
q6:    Sezione Critica
q7:    a=1;
     }
```

Figura 1: Algoritmo di mutua esclusione

Consideriamo l'algoritmo di mutua esclusione proposto alla figura 1.

1. Disegnare l'inizio dello diagramma degli stati per questo algoritmo (massimo 10 stati).
2. Verifica questo algoritmo la proprietà di mutua esclusione? Giustificare.
3. Verifica questo algoritmo la proprietà di assenza di deadlock? Giustificare.
4. Verifica questo algoritmo la proprietà di assenza di starvation? Giustificare.

1)



2)

No, non è verificata. Prendiamo in considerazione la seguente esecuzione:

P	Q	a
p1	q1	0
p2	q1	0
p6	q1	1
p6	q2	1
p6	q6	2

3) affinchè si verifichi deadlock, entrambi i processi dovrebbero bloccarsi nei rispettivi while p5 e q5. L'unico momento in cui $a \neq 1$ e $a \neq 2$ è quando $a=0$, ma per arrivare a p6 e q5, i processi passano sempre da p2 e q2: quindi a sarà sempre 0 1 o 2.

4) L'assenza di starvation non viene verificata. Se lo scheduler desse priorità solo a P, dopo che Q è arrivato in attesa nel ciclo q5, e P continua all'infinito, esegue p7 (a=2), ma poi riesegue p1 e p2 reinposta a=1, allora Q rimarrà bloccato.

2 Esercizio 2

In uno stadio, ci sono vari giocatori di calcio, alcuni appartengono alla squadra A e altri alla squadra B e vogliono disputare varie partite. Durante una partita si gioca a cinque contro cinque (esattamente), poi, una volta finita, può iniziare una nuova partita. Per iniziare una nuova partita, i giocatori devono aspettare che quelli che hanno partecipato all'ultima partita abbiano finito. Il comportamento di ogni giocatore è il seguente ciclo:

1. Aspetta finché riesce a partecipare ad una partita.
2. Partecipa ad una partita

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, tenendo in considerazione che vogliamo un processo leggero per ogni giocatore (quindi un codice per i giocatori A ed uno per i giocatori B).

```
//variabili
int numA=0;
int numB=0;
mutex l;
statopartita=ENTRARE;      //{ENTRARE,IN_CORSO}
varcond entra;
varcond gioca;

//Giocatore A
while(true){
   lock(l);
    while(numA==5 || partita != ENTRARE){
        wait(l,entra);
   }
    numA++;
   if(numA==5 && numB==5){
       partita=IN_CORSO;
       broadcast(l,gioca);
   }else{
       while(partita != IN_CORSO){
           wait(l,gioca);
       }
   }
   unlock(l);

   lock(l);
   numA--;
   if(numA==0 && numB==0){
       partita=ENTRARE;
       broadcast(l,entra);
   }
   unlock(l);
}
```

Il codice del giocatore B è identico, bisogna cambiare numA con numB.

3 Esercizio 3

Consideriamo l'algoritmo con message passing proposto sulla figura 2 per 2 processi. Assumiamo che il codice fra `[...]` si esegue in modo atomico sul sito di ciascun processo.

1)

1. Descrivere in qualche righe come funziona questo algoritmo.
2. Verifica questo algoritmo la proprietà di mutua esclusione? Giustificare.
3. Verifica questo algoritmo l'assenza di deadlock? Giustificare.
4. Verifica questo algoritmo la proprietà di assenza di starvation? Giustificare.

```
--Process Controller
    int t=0;
    while(true){
c1: }

--Gestione dei messaggi del Controller
case (req,i,t2):
  [if(t2==t){
      send(ok)->Pi;
      t=t+1;}
   else{
     send(nok)->Pi;
  }]
  case out:
    t=t+1;
```

```
--Process Pi per i in {0,1}
   boolean CS=false;
   int t=0;

    while(true){
p1:   Sezione Non Critica
p2:   while(CS==false){
p3:     send(req,i,t)->Controller;}
p4:   Sezione Critica
p5:   send(out)->Controller;
p6:   CS=false;
p7:   t=t+2;
}

--Gestione dei messaggi di Pi
--per i in {0,1}
case(nok):
  t=t+2;
case(ok):
  CS=true;
```

Figura 2: Algoritmo di mutua esclusione con message passing

4 Esercizio 4

In una città, c'è un ponte che può essere attraversato in due direzioni, da est a ovest e da ovest a est, ma non possono viaggiare sul ponte macchine che vanno in direzioni opposte allo stesso tempo. Inoltre, non possono esserci più di cinque macchine sul ponte in un dato momento. Il comportamento di un'auto è il seguente:

1. Aspetta di poter passare sul ponte
2. Passa sul ponte
3. Esce dal ponte

Il comportamento del ponte è il seguente (inizialmente, lascia passare le macchine che vanno da ovest a est) in ciclo:

1. Aspetta che il ponte sia vuoto
2. Cambia la direzione del ponte

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, tenendo in considerazione che vogliamo un processo leggero per ciascun tipo di macchina (per ciascuna direzione) e per il ponte.

```
//variabili
int nAuto = 0;
lock l;
direzione = OVESTEST;   {ESTOVEST, OVESTEST}
varcond passare


//auto
while (true){
      lock(l);
      while ( nAuto == 5 || DIREZIONE != mia_direzione){
          wait (l, passare);
      }
      nAuto++;
      unlock(l);

      lock(l);
      nAuto --;
      broadcast (l, passare);
      unlock(l);
}

//ponte
while(true){
    lock(l);
    while(nAuto != 0) {
      wait(l,passare);
    }
    if ( direzione == OVESTEST){
        direzione = ESTOVEST;
    } else {
        direzione = OVESTEST;
    }
    broadcast (l, passare);
    unlock(l);
}
```